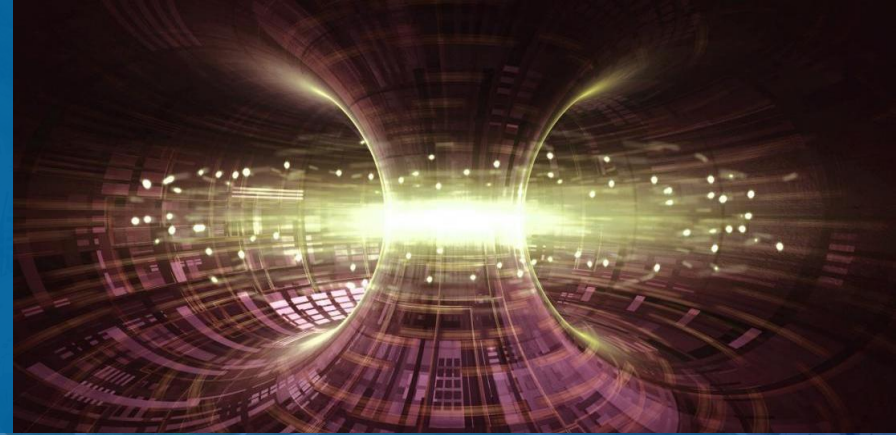


July 29, 2026

Multi-Agent AI System for PETSc Simulation of Plasma in a Tokamak (for Nuclear Fusion Energy)



Presenter

Sarthak Sharma^{1,2}
Intern and PhD student

Advisors

Dr Junchao Zhang¹, Dr Lois Curfman McInnes¹, Prof Matthew Knepley².

¹Division of Mathematics and Computer Science, Argonne National Laboratory,

²State University of New York at Buffalo.



U.S. DEPARTMENT
of ENERGY

Argonne National Laboratory is a
U.S. Department of Energy laboratory
managed by UChicago Argonne, LLC.



Contents

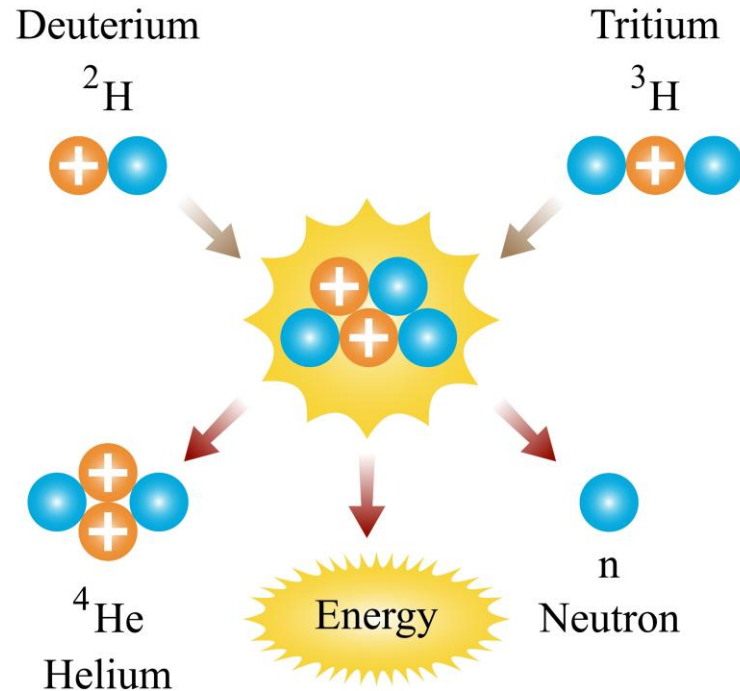
1. Why nuclear fusion energy needs trustworthy simulation
2. The modeling challenge
3. The problem: Grad–Shafranov equilibrium
4. The system: 3-layer multi-agent AI pipeline
5. The AI pipeline in action
6. AI-agent-generated PETSc solver (excerpt)
7. Verification: it is actually right
8. Decision-gate metrics
9. Fully autonomous: the built-in orchestrator
10. What we built and contributed
11. Takeaways and next steps

Why nuclear fusion energy needs trustworthy simulation

Nuclear fusion could be a near-limitless clean energy source, if we can solve some challenges.

- Combine light nuclei to create heavy nuclei (e.g., hydrogen isotopes converted to helium).
- Small amount of mass converted to large amount of energy:
$$E = mc^2$$
- Source of energy in stars.

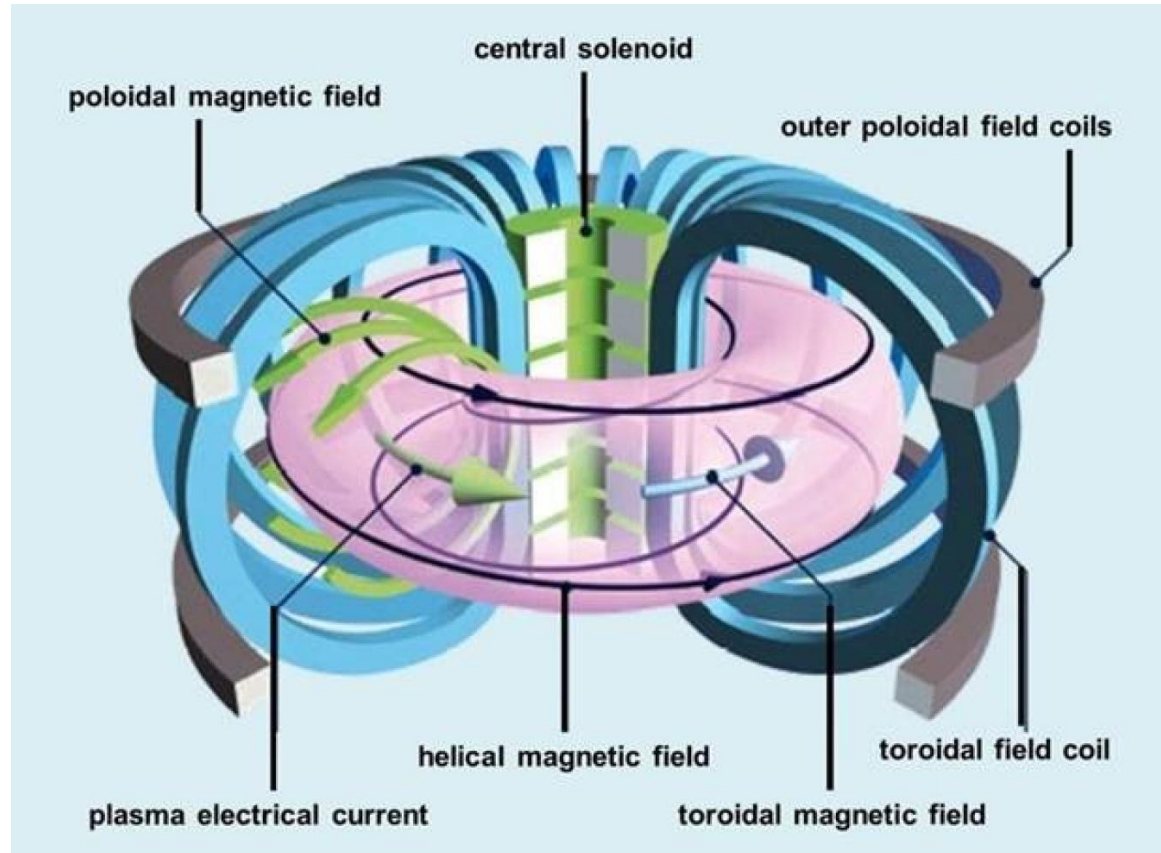
Nuclear Fusion



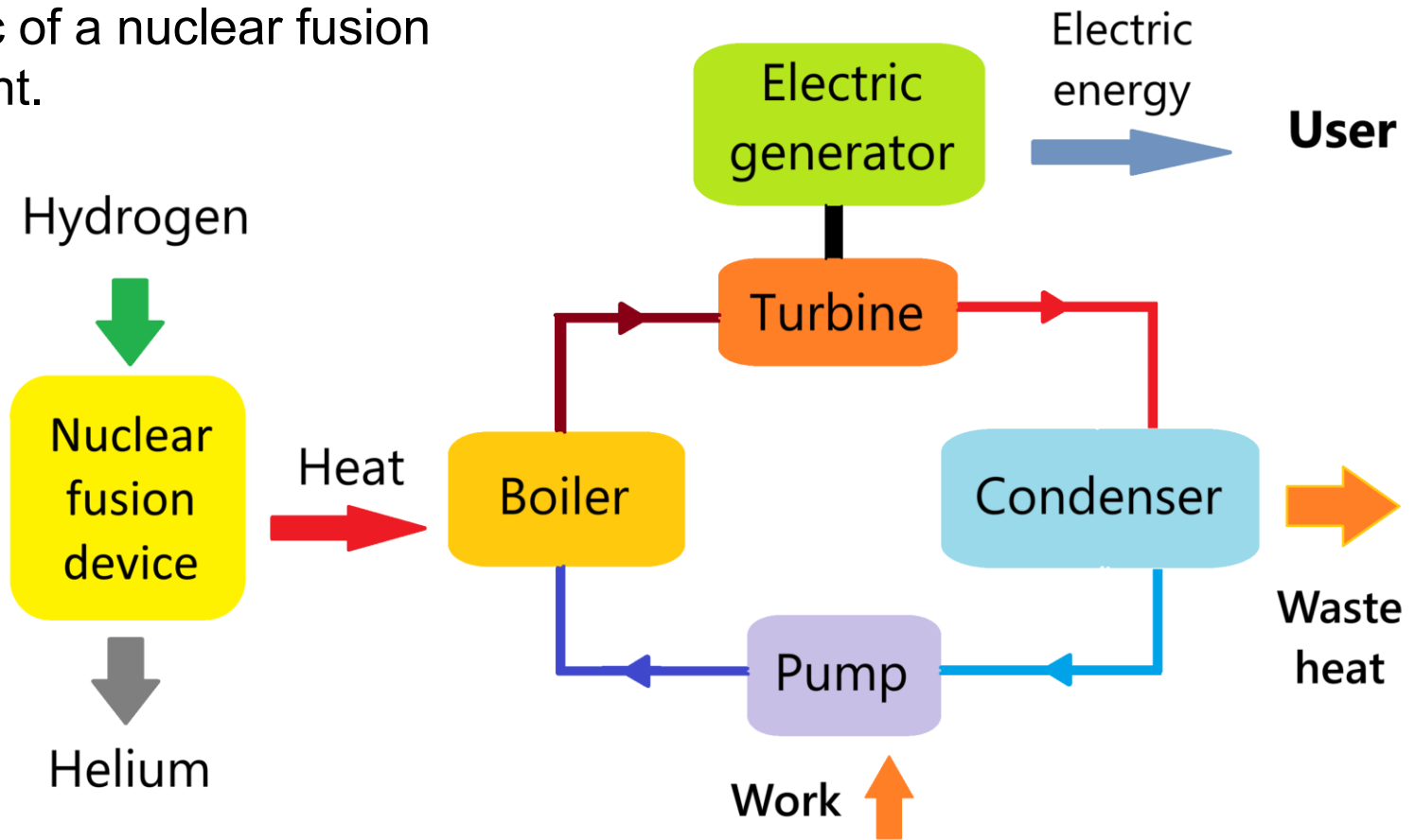
Deuterium–tritium fusion

Why nuclear fusion energy needs trustworthy simulation (continued)

- Tokamak confines plasma (temperature around 150 million °C) in a magnetic “cage” long enough to fuse.
- Accurate and scalable simulations needed before expensive and time-consuming physical experiments.
- Simulations must be trustworthy. Plausible-looking wrong answer is worse than no answer.



Schematic of a nuclear fusion power plant.



The modeling challenge

Turning fusion physics into correct and fast HPC code needs scarce and implicit expertise.

- **Numerical methods:** Choose grids, discretizations, and solvers that actually converge.
- **Library:** Express it in PETSc (Portable, Extensible Toolkit for Scientific Computation) a library for high-performance computing.
- **Parallelism:** Run on many CPU cores / GPUs.
- **Verification:** Prove the computed answer is right, not merely plausible.
- **Question:** can a multi-agent AI system automate this whole path, and can we trust it?

The problem: Grad–Shafranov equilibrium

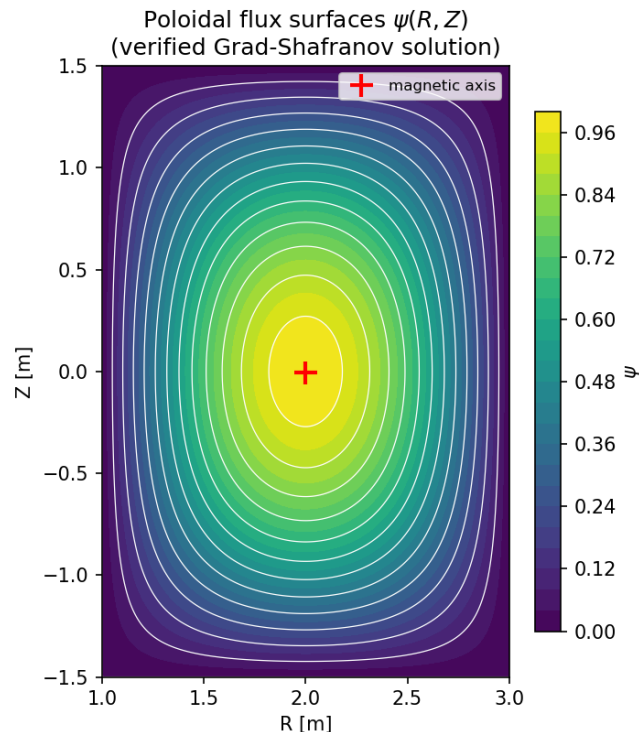
Axisymmetric ideal-MHD force balance for the poloidal flux $\psi(R, Z)$.

- Equilibrium magnetohydrodynamics for 2-dimensional plasma in a tokamak:

$$\frac{\partial^2 \psi}{\partial r^2} - \frac{1}{r} \frac{\partial \psi}{\partial r} + \frac{\partial^2 \psi}{\partial z^2} = -\mu_0 r^2 \frac{dp}{d\psi} - \frac{1}{2} \frac{dF^2}{d\psi}$$

where ψ is magnetic flux, p is plasma pressure, μ_0 is magnetic permeability, $F = rB_\theta$, and (r, θ, z) are the cylindrical coordinates.

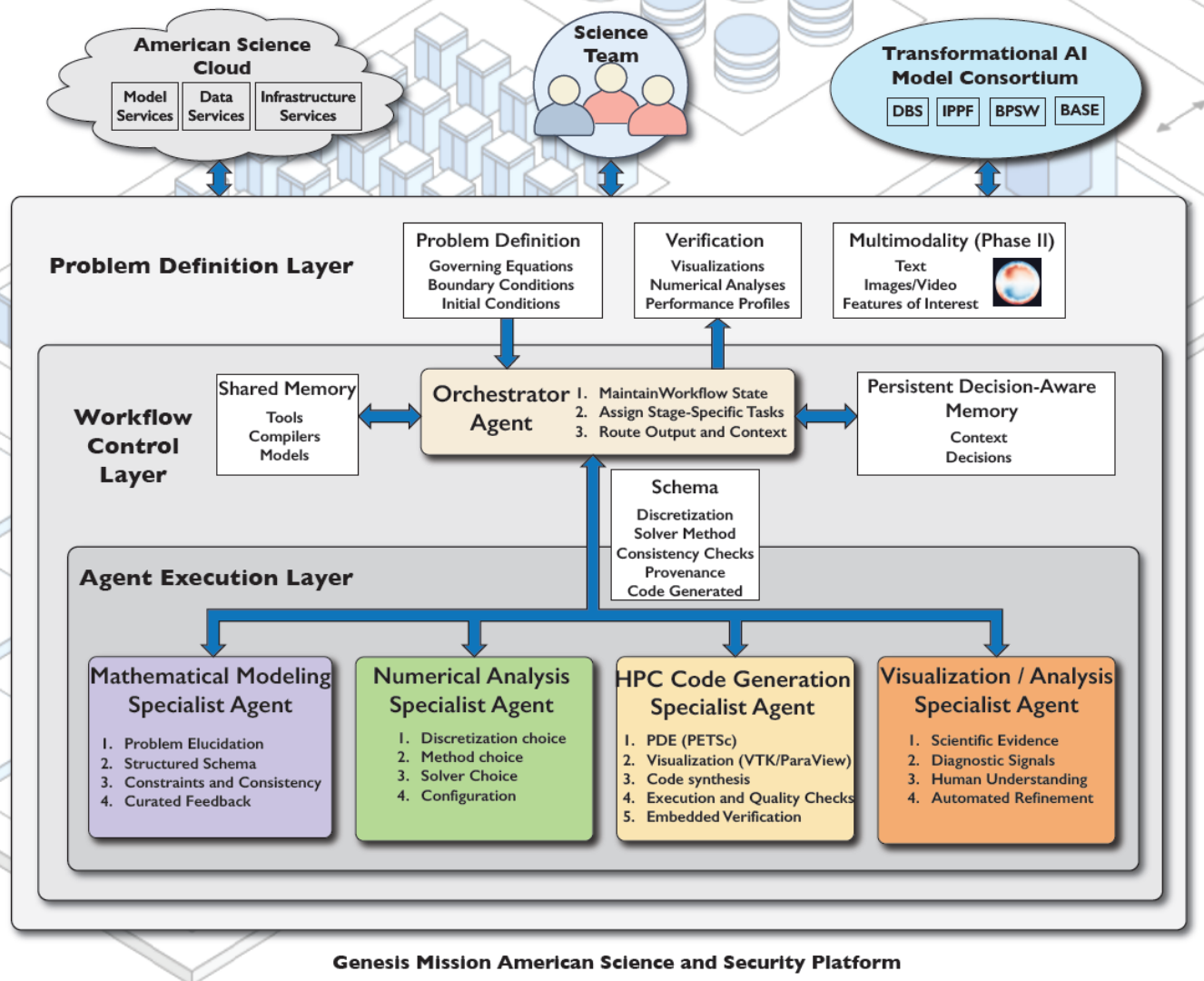
- Elliptic, non-linear, time-independent.
- Its solution gives the flux surfaces, magnetic axis, and safety factor q .
- A natural problem for PETSc SNES (Scalable Non-linear Equations Solvers).
- Verifiable: a manufactured exact solution gives a rigorous convergence test.



The system: 3-layer multi-agent AI pipeline

PETSc Model-Context-Protocol (MCP) agents, coordinated end to end.

- Problem definition (**Mathematical Modeling agent**) → strong / weak form, name, time-dependence.
- Agent execution (**Numerical Analysis agent**): grid, discretization, solver (SNES).
- Agent execution (**HPC Code Generation agent**): writes, compiles and runs PETSc C programs.
- Execution substrate (**Compile and Run agent**): make / run on the real machine.
- All agents run against Argonne's Argo gateway (Claude Opus 4.8).
- A project-owned driver records every artifact with provenance (reproducible and resumable).



The AI pipeline in action

From one English prompt to a compiled, running solver, with no human edits.

- **Prompt:** *“the Grad-Shafranov equilibrium for the plasma in a tokamak.”*
- **Modeling AI agent:** identified ‘Grad-Shafranov equation’; time-dependent = False.
- **Numerical Analysis AI agent:** non-linear solve with SNES on a structured grid.
- **Code Generation AI agent:** 267-line PETSc program (DMDA + SNES + true Jacobian).
- Compiled and ran on 1 and 4 MPI ranks. No human edits.

AI-agent-generated PETSc solver (excerpt)

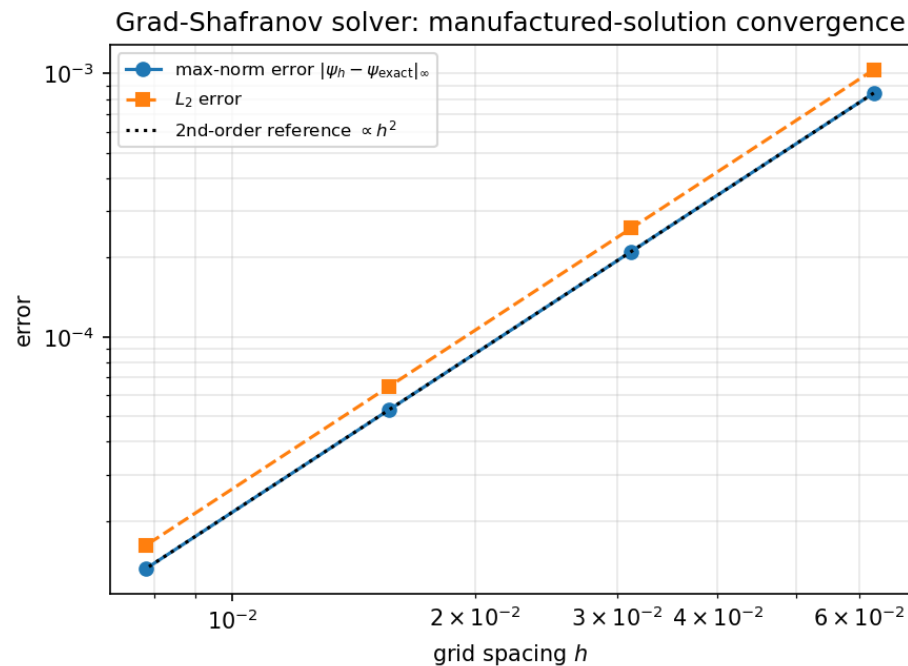
A true-Jacobian finite-difference SNES residual, written by the Code Generation AI agent.

```
static PetscErrorCode FormFunction(SNES snes, Vec X, Vec F, void *ctx){
    /* Delta*_h psi = (psi_E-2psi_C+psi_W)/hR^2
                       - (1/R)(psi_E-psi_W)/(2hR)
                       + (psi_N-2psi_C+psi_S)/hZ^2 */
    f[j][i] = dRR - dR1/R + dZZ - ForcingF(user,R,Z); /* residual */
}
SNESSetFunction(snes, r, FormFunction, &user);
SNESSetJacobian(snes, J, J, FormJacobian, &user); /* true Jacobian */
SNESSolve(snes, NULL, x);
```

Verification: it is actually right

Method of manufactured solutions: a measured order of accuracy, not just plausible output.

- Known exact ψ : check the numerical error as the grid refines.
- Max-norm error = 1.32×10^{-5} on the finest 257^2 grid.
- Observed order of accuracy $p = 2.00, 2.00, 2.00$ (expected 2 for central differences).
- SNES converged:
CONVERGED_FNORM_RELATIVE.



Manufactured-solution convergence

Decision-gate metrics

Correctness, human effort, and efficiency: the numbers behind the demo.

- Correctness:
 - model identified,
 - C programs compiled and ran,
 - second-order convergence.
- Human effort: 0 lines of solver code hand-written. It was all AI-generated.
- Efficiency: total wall-clock ≈ 450.8 seconds; code-gen used 5 tool calls.
- Around 23 completions of LLM (large language models) across the whole pipeline.

Fully autonomous: the built-in orchestrator

The shipped LLM orchestrator drives all four agents itself (no Python driver).

- Given only the prompt, an inner Claude agent sequenced all four MCP servers on its own.
- It independently chose a Solov'ev Grad–Shafranov model and a DMPLEX + PetscFE finite-element solve.
- The generated code compiled cleanly and ran: 225 DOFs, L2 norm $\psi = 2.04$, return code 0.
- It surfaced a real upstream bug: a hard-coded iteration cap flagged a converged run as failure.
- We fixed it and contributed the patch; the re-run finished: “I have completed the orchestration.”

What we built and contributed

Reusable tooling around the agents, plus fixes contributed back upstream.

- A project-owned orchestration driver with full artifact provenance (resumable).
- Verification + post-processing (convergence, flux surfaces) and a metrics harness.
- Four fixes contributed back to the open PETSc multi-agent system:
 - CWD-independent server resolution; portable stdio interpreter.
 - graceful operation without the documentation / RAG services.
 - code-generation and orchestrator loops that reliably capture results.

Takeaways and next steps

A verification-driven multi-agent path from prompt to trustworthy HPC code

- A multi-agent AI system generated a correct, verified PETSc fusion-MHD solver from a prompt.
- Both the Python driver and the fully-autonomous orchestrator succeeded end to end.
- Verification-driven: convergence + exact-solution checks, not just plausible output.
- Next: shaped real-machine equilibria (Solov'ev / Cerfon–Freidberg), q-profile, GPU scale-up.
- Code + docs: github.com/engineer-scientist/petsc_mcp_servers_tokamak

References and acknowledgements

- AI coding agent: *Claude Code* by Anthropic (<https://claude.com/product/claude-code>)
- Advisors: Dr Junchao Zhang, Dr Lois Curfman McInnes, Prof Matthew Knepley.
- *Automated Problem-to-Solution Generation for PDE-Based Simulation Science*, Dr Lois Curfman McInnes, Dr Junchao Zhang, Dr Matthew Knepley, et al (2026).
- *PETSc MCP Servers*, Barry Smith (2026): https://gitlab.com/petsc/petsc_mcp_servers
- *Grad-Shafranov equation* (https://en.wikipedia.org/wiki/Grad-Shafranov_equation).
- ANL CELS General Computing Environment: <https://help.cels.anl.gov/docs/linux/>

Argonne
NATIONAL LABORATORY



U.S. DEPARTMENT
of ENERGY